

Documentオブジェクト

前回、**window** オブジェクト はブラウザ全体の「社長さん」だと説明しましたね。今回の **document** オブジェクト は、**window** オブジェクトの下にあるプロパティで、その社長さんの下で働く「現場監督（げんばかんとく）」です。

[!IMPORTANT]

オブジェクトは別のオブジェクトのプロパティになることがあります。

学生の方は、「ウェブページという建物を管理し、中身を書き換えたり、新しくパーツを作ったりする責任者」と考えると、イメージが湧きやすくなるかもしれません。

documentオブジェクトとは、ブラウザに表示されているHTML（ウェブページの中身）そのものを指します。

JavaScriptを使って、文字の色を変えたり、ボタンが押されたときに動きをつけたりするのは、すべてこの「現場監督（document）」に命令を出しているからです。

これを専門用語で **DOM（ドム / Document Object Model）** と呼びます。HTMLを「木の枝」のような構造として管理しているのが特徴です。

DOMツリーについては、次のセクションで、くわしく説明します。ここでは、Windowとdocumentの違いをしっかりと頭に入れてください。

1. documentオブジェクトとは？

ブラウザに表示されているHTML（ウェブページの中身）そのものを指します。

JavaScriptを使って、文字の色を変えたり、ボタンが押されたときに動きをつけたりするのは、すべてこの「現場監督（document）」に命令を出しているからです。

これを専門用語で **DOM（ドム / Document Object Model）** と呼びます。HTMLを「木の枝」のような構造として管理しているのが特徴です。

2. よく使うプロパティ（ページの情報）

現場監督が知っている、今のページの情報は、

- **document.title**

ブラウザのタブに表示されている「ページのタイトル」です。JavaScriptで書き換えることもできます。

- **document.body**

HTMLの `<body>` タグの中身全体です。ページ全体の背景色を変えたいときなどに使います。

- **document.URL** ^{げんざいひら}現在開いているページの住所（URL）^{おし}を教えてください。

3. よく使うメソッド（現場監督への命令）

これが一番大切です！ ^{いちばんたいせつ}特定の場所を探したり、^{とくてい}中身を変えたりする命令です。

^{ばしょ}場所を探す（セレクト）

- **getElementById("ID名")** ^{とくてい}特定のIDがついたパーツを1つだけ探します。^{さが}一番速くて正確です。^{いちばんはや}
- **querySelector("セレクト")**
CSSと同じ書き方（^{おな}**.class** や ^か**#id**）でパーツを探せます。^{さが}とても便利です！^{べんり}

これらは、以前にも出てきましたね。実はdocumentオブジェクトのメソッドだったのです！

[!note]

document.getElementById("ID名") のように書いてもかまいませんが、^{ふつう}普通は**document.**を省略して、**getElementById("ID名")**と書きます。

^{なかみ}中身を書き換える・^{つく}作る

- **createElement("タグ名")** ^{あた}新しいパーツ（**<div>** や **** など）をメモリの中に作ります。^{なか}
- **write("文字")**
ページに直接文字を書き込みます（※あまり使いませんが、^{きほん}基本として覚えておきましょう）。

4. 実際に動かしてみよう！

Visual Studio Code を使って、^{つか}以下のようなhtmlを作成しましょう。^い名前は**doc_obj.html**としましょうか。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h1 id="greeting">こんにちは！</h1>
  <br>
  <p>コンソールを開いて、下のJavaScriptを入力してみましょう。</p>
  <br>
  <pre>
    // 1. IDが "greeting" というパーツを探す
    const message = document.getElementById("greeting");
```

```
// 2. その中身を書き換える
message.textContent = "こんにちは、日本へようこそ!";

// 3. スタイル（色）も変えちゃう
message.style.color = "red";
</pre>
<br>
</body>
</html>
```

3. よく使うメソッド（現場監督への命令）

これが一番大切です！ 特定の場所を探したり、中身を変えたりする命令です。

場所を探す（セクタ）

- `getElementById("ID名")` 特定のIDがついたパーツを1つだけ探します。一番速くて正確です。
- `querySelector("セクタ")`
CSSと同じ書き方（`.class` や `#id`）でパーツを探せます。とても便利です！

これらは、以前にも出てきましたね。実はdocumentオブジェクトのメソッドだったのです！

[!note]

`document.getElementById("ID名")` のように書いてもかまいませんが、普通は `document.` を省略して、`getElementById("ID名")` と書きます。

中身を書き換える・作る

- `createElement("タグ名")` 新しいパーツ（`<div>` や `<li>` など）をメモリの中に作ります。
- `write("文字")`
ページに直接文字を書き込みます（※あまり使いませんが、基本として覚えておきましょう）。

4. 実際に動かしてみよう！

Visual Studio Code を使って、以下のようなhtmlを作成しましょう。名前は `doc_obj.html` としましょうか。
保存できたら、Visual Studio Code 上から、`Open with Live Server` を使って、開きましょう。

`../src/img/doc_methodを開く.jpg`

ブラウザに表示された指示に従って、コンソールにJavaScriptを入力しましょう。

コンソールを開くには、F12 キー（ノートPCは Fn + F12） または Ctrl + Shift + I を使います。

オブジェクトと `const` の意外な関係

上のコードを見て、疑問に思いませんか？「`const` で宣言された変数は、後から値を変更できないはずでは？」そう思った人は、変数の説明をととてもよく覚えている人です。

しかし、上のコードはエラーになりません。それについて、説明しましょう。

JavaScriptの初心者が混乱しやすい点のひとつが、オブジェクトと `const` の意外な関係です。

「`const` で宣言した変数は中身を変えられない」と説明しましたが、オブジェクトの場合は少し違います。

- 名札の固定（参照の不変性）：`const` で作ったオブジェクトそのものを別のオブジェクトに入れ替えることはできません（別の箱に名札を付け替えるのは禁止）。
- 中身の変更はOK：箱の中に入っている個別のデータ（プロパティ）を書き換えることは可能です。

「強力なガムテープで名札が貼られた箱」をイメージしてください。ガムテープで固定されているので、名札を別の新しい箱に貼り直すことはできません。でも、箱のふたをひとつ開けて、中に入っているお菓子を入れ替たり、ジャムを塗ったりすることは自由なのです。

例えるなら、「ファイルの保存場所（パス）」は固定されているけれど、「ファイルの中身」は自由に書き換えられるようなイメージです。

先ほどの例では、`const` を使って宣言した `message` という箱を開けて、中身を「こんにちは、日本へようこそ！」に入れ替えました。これはエラーにはなりません。

しかし、

```
// 1. IDが "greeting" というパーツを探す
const message = document.getElementById("greeting");

// 2. タグがpのパーツを探す
message = document.querySelector("p")
```

はエラーになります。`message` というラベルを「"greeting"というIDの箱」からはがして、「"p"というタグの別の箱」に貼ろうとしているからです。

`const` は `message` というラベルと、それを入れる入れ物との関係を固定して、ラベルの張替えを許しません。

しかし、あくまでラベルと入れ物との関係であって、入れ物が同じであれば、その中身が変わっても、かわらないのです。

5. 学生へのアドバイス：`window` と `document` の使い分け

「どっちを使えばいいの？」と聞かれたら、こう答えてあげてください。

「ブラウザの機能（タイマー、アラート、URL移動）を使いたいときは **window**」

「ページの中身（文字、画像、ボタン）をいじりたいときは **document**」

まとめテーブル

めいれい じょうほう なまえ 命令・情報の名前	やくわり 役割	たと ばなし 例え話
title	ページのタイトル	ほん ひょうし なまえ 本の表紙の名前
getElementById()	とくてい パーツを指名する	しゅっせきばんごう ばん ひと よ 「出席番号〇番の人！」と呼ぶ
querySelector()	じゆう 自由にパーツを探す	あか ふく き ひと さが 「赤い服を着ている人！」と探す
textContent	も じ か か 文字を書き換える	かんばん も じ か なお 看板の文字を書き直す
createElement()	あた たら よう そ つく 新しい要素を作る	あた たら よう い 新しいレンガを用意する

いかがでしょうか？「現場監督の document くん」がいれば、ウェブページを思った通りに操れるようになります。

オブジェクトと **const** の意外な関係

上のコードを見て、疑問に思いませんでしたか？「**const** で宣言された変数は、後から値を変更できないはずでは？」そう思った人は、変数の説明をととてもよく覚えている人です。

しかし、上のコードはエラーになりません。それについて、説明しましょう。

JavaScriptの初心者が混乱しやすい点のひとつが、オブジェクトと **const** の意外な関係です。

「**const** で宣言した変数は中身を変えられない」と説明しましたが、オブジェクトの場合は少し違います。

- 名札の固定（参照の不変性）：**const** で作ったオブジェクトそのものを別のオブジェクトに入れ替えることはできません（別の箱に名札を付け替えるのは禁止）。
- 中身の変更はOK：箱の中に入っている個別のデータ（プロパティ）を書き換えることは可能です。

「強力なガムテープで名札が貼られた箱」をイメージしてください。ガムテープで固定されているので、名札を別の新しい箱に貼り直すことはできません。でも、箱のふたをひとつ開けて、中に入っているお菓子を入れ替たり、ジャムを塗ったりすることは自由なのです。

例えるなら、「ファイルの保存場所（パス）」は固定されているけれど、「ファイルの中身」は自由に書き換えられるようなイメージです。

先ほどの例では、**const** を使って宣言した **message** という箱を開けて、中身を「こんにちは、日本へようこそ！」に入れ替えました。これはエラーにはなりません。

しかし、

```
// 1. IDが "greeting" というパーツを探す
const message = document.getElementById("greeting");

// 2. タグがpのパーツを探す
message = document.querySelector("p")
```

はエラーになります。`message` というラベルを「"greeting"というIDの箱」からはがして、「"p"というタグの別の箱」に貼ろうとしているからです。

`const` は `message` というラベルと、それを入れる入れ物との関係を固定して、ラベルの張替えを許しません。

しかし、あくまでラベルと入れ物との関係であって、入れ物が同じであれば、その中身が変わっても、かまわないのです。

5. 学生へのアドバイス：`window` と `document` の使い分け

「どっちを使えばいいの？」と聞かれたら、こう答えてあげてください。

「ブラウザの機能（タイマー、アラート、URL移動）を使いたいときは `window`」

「ページの中身（文字、画像、ボタン）をいじりたいときは `document`」

まとめテーブル

めいれい 命令・情報の名前	やくわり 役割	たと ばなし 例え話
<code>title</code>	ページのタイトル	本の表紙の名前
<code>getElementById()</code>	特定のパーツを指名する	「出席番号〇番の人！」と呼ぶ
<code>querySelector()</code>	自由にパーツを探す	「赤い服を着ている人！」と探す
<code>textContent</code>	文字を書き換える	看板の文字を書き直す
<code>createElement()</code>	新しい要素を作る	新しいレンガを用意する

いかがでしょうか？「現場監督の `document` くん」がいれば、ウェブページを思った通りに操れるようになります。## プロパティとメソッド

`document` オブジェクトは、ブラウザの画面に映っている「白い紙（HTMLの文章）」そのものです。これを使うことで、HTMLの文字を読み取ったり、新しく書き換えたりすることができます。

留学生の方に分かりやすいよう、こちらもプロパティ（情報）と**メソッド（命令）**に分けて整理しました。

1. documentオブジェクトのプロパティ（情報）

プロパティは、今^{いま}見ているWebページの「中^{なか}身^みや設^せ定^{てい}のデ^でー^た」です。

📄 ページの中身（HTMLのパーツ）

- **body**（ボディ）：
- **意味**：HTMLの `<body>` タグの部分、つまり「ユーザーの目に見える画面全体」を指すオブジェクトです。ここに背景の色を変える命令などを出せます。
- **forms**（フォームズ）：
- **意味**：ページの中にあるすべての `<form>` タグの「詰め合わせセット（リスト）」です。

[!note] 入^{にゅう}力^{りき}一^{いっ}覧^{らん}にあった **forms** は、大^お文^お字^{もじ}小^こ文^{もじ}字^じのタイポ（打^うち間^ま違^{ちが}い）になりやすいポイントです。正^{ただ}しくは **forms**（rのあとにo）だと教えてあげてください。

🌐 ネットワークやURLの情報

- **URL**（ユーアールエル）：
- **意味**：今見ているページの「完全^{かんぜん}なURL（ア^あド^どレ^れス）」の文字列です。
- **domain**（ドメイン）：
- **意味**：URLの中から、サーバーの「名^な前^{まえ}（場^ば所^{しょ}）」の部分だけを抜き出したものです（例: `google.com` や `yahoo.co.jp`）。
- **location**（ロケーション）：
- **意味**：`window.location` と同じ、URLの詳^{くわ}しい情^{じょう}報^{ほう}が入^{はい}ったオブジェクトです。
- **referrer**（リファラー）：
- **意味**：ユーザーが「どこのページ（リン^もク^と元^{げん}）からこのページに飛^とんできたか」という前^{まえ}のページのURLを教^{おし}えてくれます。

🔧 ページの設定・状態

- **title**（タイトル）：
- **意味**：HTMLの `<title>` タグに書^かいた、ブラウザの「タブ」に出^でるページのタイトルです。JavaScriptから `document.title = "新しい名前";` と書^かけば、途^と中^{ちゅう}でタブの名前を変^なえ^まえ^かられます。
- **lastModified**（ラスト・モディファイド）：
- **意味**：このHTMLファイルが「最^{さい}後^ごに更^{こう}新^{しん}（セ^さーブ）され^されたのはいつか」という日^{にち}時^じ（日^ひ付^{つけ}と時^じ間^{かん}）を教^{おし}えてくれます。

- **cookie** (クッキー) :

- **意味** : ブラウザにユーザーのログイン状態や設定を一時的に保存しておくための「小さなメモ帳」のようなデータです。

2. documentオブジェクトのメソッド (命令)

 **画面に文字を書く (現在はあまり使われません)**

- **write("文字") / writeln("文字")** :
- **意味** : 画面に直接文字を書き出す命令です。writeln は最後に自動で改行 (改行コード) を入れてくれます。

[!note] これらは初期のJavaScriptでよく使われましたが、現在は `document.createElement()` などの新しい書き方が主流です。ページが完全に読み終わった後に `document.write()` を実行すると、今ある画面の中身が全部消えて真っ白になってしまうという危険な罠があるため、留学生には「こういう古い歴史の命令もあるよ」と紹介する程度が良いです。

 **文字を書くための「紙」を開く・閉じる**

- **open() / close()** :
- **意味** : 上の `document.write()` で文字を書くために、「新しいドキュメントの紙を開く」命令と、書き終わったから「紙を閉じて画面を完成させる」命令です。

[!note] `window.open()` / `close()` は「ブラウザのウィンドウ (タブ)」を開き閉めしますが、`document.open()` / `close()` は「ページの中の文章 (テキストデータ)」を開き閉めするので、ごっちゃにしないよう注意が必要です。

3. 留学生向けのまとめ

Webページを支配する document

`document` オブジェクトは、ブラウザに映っているWebページの「白い紙そのもの」です。

- * **プロパティ (情報)** : `document.title` を使えばタブの名前が分かり、`document.body` を使えば画面全体の色やスタイルを変えることができます。また、`forms` を使えばページの中にある入力ボックスを全部集めることができます。
- * **メソッド (命令)** : `write()` などの画面に直接文字を書く古い命令もありますが、今のプログラミングではあまり使いません。

これからWebサイトの文字を書き換えたり、ボタンが押されたときの動きを作ったりするときは、必ずこの `document` からスタートします。JavaScriptで一番たくさん使う大黒柱のようなオブジェクトなので、しっかり仲良くなりましょう！

いかがでしょうか。歴^れ史^きが長^{なが}いオブジェクトなので古^{ふる}いメソッドも混^まざっていますが、**body**、**forms**、**title**、**URL** などのプロパティは今^{いま}でも毎^{まい}日^{にち}のように使^{つか}う主^{しゅ}役^{やく}たちです。